Agent-Computer Observation Interfaces Enable Dynamic Computer Use

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

SWE-agent [Yang et al., 2024] established the *action* interface as an underexplored design axis for software-engineering agents. We make the analogous case for the *observation* interface in computer-use (CU) agents. Current CU agents—closed and open-source alike—tie observation to action: one screenshot every 3–5 s, with no audio. Between screenshots they are blind and deaf to video, animations, transient UI events, meetings, and spoken instructions.

We introduce the **Agent-Computer Observation Interface (AOI)**, a model-agnostic perception layer that decouples continuous, adaptive observation from discrete actions. It has three gated components—inter-step keyframe capture, volume-gated audio transcription, and CU-model-generated visual narration that persists as text—each of which produces almost nothing on static, silent content, reducing to the standard loop without degrading it.

On **DynaCU-Bench** (100 dynamic browser tasks plus a 50-task static control), eight CU models from 7B to frontier scale gain +17 to +48 pp over their screenshot baselines with zero retraining (paired McNemar $p < 10^{-3}$ for all); Claude Sonnet 4.6 reaches a 3-seed mean of 78.7%. On a 12-task audio subset, AOI agents score 12/12 where production streaming voice baselines largely fail (OpenAI Realtime [OpenAI, 2024] 3/12, Gemini Live [Google, 2025] 0/12). Three findings reframe CU perception: keyframe *selection* is unimportant—value emerges only once captured frames are narrated into persistent text; narration helps mainly by making the model *articulate* what it sees; and the interface is not a fixed bundle—on Gemini 3 Flash the keyframe stream regresses −12 pp from image-token dilution, so components must be tuned per model rather than shipped as one configuration.

Code: `https://github.com/19PINE-AI/AOI`
Website: `https://01.me/research/AOI`

1 Introduction

Computer-use agents are blind between screenshots and deaf to audio. A computer-use (CU) agent perceives the screen as a periodic stream of static screenshots—one every 3–5 seconds—and has no audio channel at all (Figure 1). Between snapshots it is *blind*: a video plays, a slide animates, a toast notification appears and auto-dismisses, all unobserved. And it is *deaf*: meeting speech, a spoken instruction, a notification chime never reach the model. An entire class of everyday computer use—watching a screencast, following a meeting, answering a voice prompt, reacting to a transient dialog—is therefore out of reach, even for agents that handle static interfaces well.

The gap is real and unaddressed. On static GUI benchmarks such as OSWorld [Xie et al., 2024], screenshot agents have climbed from sub-10% to above 50% [Andreux et al., 2025, Wang et al., 2025a], approaching the human baseline; they fill forms, navigate sites, edit documents, and drive professional software. But the dynamic-and-audible regime is documented as a standing gap and left open: video-LLM GUI agents struggle across temporal tasks [Chen et al., 2024], long-context models do *worse* with video than without [Jang et al., 2024], even paused-frame game evaluation reaches only single-digit completion [Zhang et al., 2025], and dozens of hours of temporal dynamics go unobserved by screenshot agents [Chen et al., 2025]; a recent CU survey catalogues many open challenges but not the observation modality itself [Sager et al., 2025].

The observation interface is a separable design axis. Why is this hard to fix from the model side? Retraining CU models on video is expensive, model-specific, and undemonstrated at scale; video-native models brute-force frame sampling and waste tokens on redundant static frames; native real-time multimodal APIs [Google, 2025, OpenAI, 2024] work but lock users to one provider, offer no selective perception, and require migration (we compare against them in Section 8). We take a different view, by analogy to SWE-agent [Yang et al., 2024]: its lever was not a new model but the agent-computer *action* interface—which commands the model is given, how inputs and feedback are formatted. We argue the *observation* interface is the analogous lever for CU agents. Prior work enriches *what* a single snapshot encodes—accessibility trees, element overlays, text surrogates—but leaves fixed the dimension we target: *when* the agent looks and through which senses. Our operative principle is *decoupling observation (continuous, adaptive, multimodal) from action (discrete)*: every CU agent we are aware of [Anthropic, 2024a, OpenAI, 2025, Google DeepMind, 2025, Wang et al., 2025a,b, Awadallah et al., 2025, Song et al., 2025] ties the two together at one static screenshot per step, and is deaf.

Our approach. The **Agent-Computer Observation Interface (AOI)** is a lightweight perception layer between the environment and any image-based CU model. It converts continuous screen and audio streams into the sparse images and text the model already accepts, through three components, each guarded by a fast gate: inter-step keyframe capture, volume-gated audio transcription, and CU-model-generated visual narration that persists as text after the source images are pruned. On static, silent content the gates produce almost nothing and behaviour reverts to the standard loop; on dynamic content the model additionally sees what moved and heard what was said. No retraining is required—though, as we show, which components help is model-specific.

Contributions.

1. We frame the **observation interface** as a separable CU design axis, complementary to SWE-agent's action interface, with *decoupling observation from action* as the operative principle (Sections 1–3).
2. We introduce the **AOI**, a model-agnostic perception layer with three gated components, and release it together with **DynaCU-Bench** (100 dynamic tasks) and a static no-degradation control.
3. Across eight CU models from 7B to frontier scale, the AOI lifts dynamic-task success by large, significant margins with zero retraining (Section 5), while staying transparent on static work.

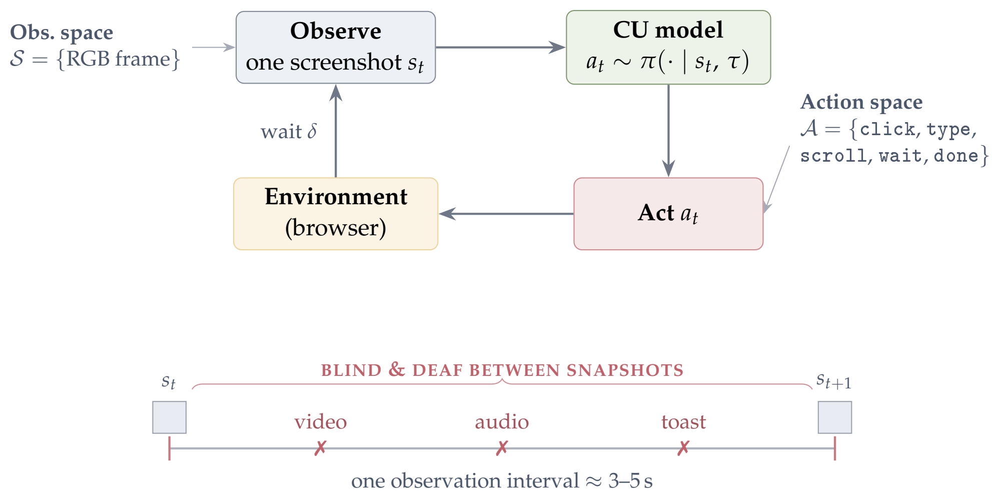


Figure 1. The computer-use agent loop and its blind spots. A CU agent repeats *observe* → *reason* → *act*: it captures a single screenshot s_t (observation space $\mathcal{S}$), the model samples an action a_t from a small grammar $\mathcal{A}$ (`click`/`type`/`scroll`/`wait`/`done`), the browser executes it, and after a buffer δ the next screenshot is taken. The interval is 3–5 s; *between* snapshots the agent is blind to anything that moves (video, animation, an auto-dismissing toast) and has no audio channel at all, so spoken content is never observed.

4. We decompose *where* the gain comes from: keyframe *selection* is immaterial, value emerges only once frames are narrated into text, and the component mix must be tuned per model—one component even flips sign on a later-generation model (Sections 6.2–7).

2 Background: The CU Agent Loop

The loop. Every current CU agent runs the same *observe* → *reason* → *act* cycle of Figure 1: capture a screenshot s_t, sample an action a_t from the model conditioned on s_t and trajectory history τ, execute it, wait a short buffer δ, repeat. Two spaces define what the agent can do. The **action space** $\mathcal{A}$ is a small grammar—click, type, scroll, wait, done—and has been the focus of prior interface work [Yang et al., 2024]. The **observation space** $\mathcal{S}$ is what we study: today it is a single RGB frame, $\mathcal{S} = \{\text{one screenshot}\}$, sampled once per action.

Why one screenshot per step leaves the agent blind. The observation interval—model inference plus action execution plus buffer—is 3–5 seconds, and the screenshot is a point sample of it. Anything that happens *between* samples is lost: the frame is static, so motion is invisible, and there is no audio path, so sound is never captured. Multi-image histories do not help—the extra images are post-action screenshots from prior steps, never the moments between them. Concretely, four kinds of content are systematically missed: (i) *transient events*—toasts and dialogs that appear and auto-dismiss within one interval; (ii) *continuous visual media*—video, animated tutorials, transitioning slides; (iii) *audio*—meeting speech, notification sounds, spoken prompts; and (iv) *periodic noise*—spinners and blinking cursors that should be *ignored* yet trip naive frame differencing. Expanding $\mathcal{S}$ to cover (i)–(iii) while suppressing (iv), without touching the model or $\mathcal{A}$, is the problem the AOI solves.

3 System Design

The AOI sits between the environment and any existing CU model, observing the entire interval between agent steps and providing the model with additional keyframes, audio descriptions, and accumulated text context—all in the standard image + text format that every CU model already accepts. For static, silent tasks, the AOI's perception gates produce nothing and behaviour reduces to the standard loop wrapped in the structured observation-record format (which does not degrade static work—Section 6.1 shows it slightly helps); for dynamic tasks, the model additionally receives the inter-step keyframes, audio transcripts, and narration. This is strictly more *information*, but not always a strictly better *outcome*: the per-step observation work adds latency, which on action-bound tasks (browser games, Section 5) can cost more than the perception gains—the AOI targets human-paced computer use, not real-time interaction. No retraining is needed.

3.1 Architecture Overview

The AOI has three components, each preceded by a fast gate (sub-millisecond) that short-circuits processing when there is nothing to perceive; Figure 2 contrasts the resulting pipeline with the standard CU loop. For a frame that passes the gates, the additional cost is dominated by CLIP-ViT-B/16 ($\sim$7 ms on GPU) for visual frames and Whisper large-v3 (variable, on demand) for audio segments; for the static, silent frames that dominate most workloads the per-frame cost is the sub-millisecond gate alone.

(1) **Inter-step keyframe capture**: continuous screen sampling at $\sim$3 Hz with gating to suppress redundant frames. Produces 0–5 keyframe images per step.
(2) **Volume-gated audio observer**: RMS energy gate followed by ASR (Whisper large-v3). Produces a text transcription per step, only when audio is present.
(3) **Visual narration context**: the CU model itself generates a brief text description of new visual information as a side-output of each inference call. These narrations accumulate in the trajectory and persist after keyframe images are pruned from context.

3.2 Inter-Step Keyframe Capture

Between agent steps, the screen may change (a video plays, a dialog appears, a slide transitions) or stay static. The AOI continuously samples the screen at $\sim$3 Hz and selects a subset of frames to include as keyframe images in the observation record (up to 5 per step). Multiple selection strategies are possible—uniform fixed-rate sampling, pixel-level differencing, random sampling, or semantic filtering via CLIP embeddings [Radford et al., 2021]. We evaluate five variants spanning these four families (uniform at 1 and 3 FPS, pixel-diff, random, CLIP) in Section 6.2 and find that all converge to similar accuracy, so the default implementation uses simple pixel-change gating: if fewer than 1% of pixels changed since the last captured frame, the sample is skipped (cost: $<$1 ms on CPU). For static screens, no keyframes are produced and the component has zero overhead. Algorithm 1 formalises the two-stage CLIP variant; the default selector drops the CLIP stage (lines 5–10) and uses only Stage 1.

3.3 Volume-Gated Audio Observation

Current CU agents have zero audio perception. They cannot hear meeting speech, notification sounds, or error alerts. Pure ASR models (Whisper) transcribe speech but produce nothing for

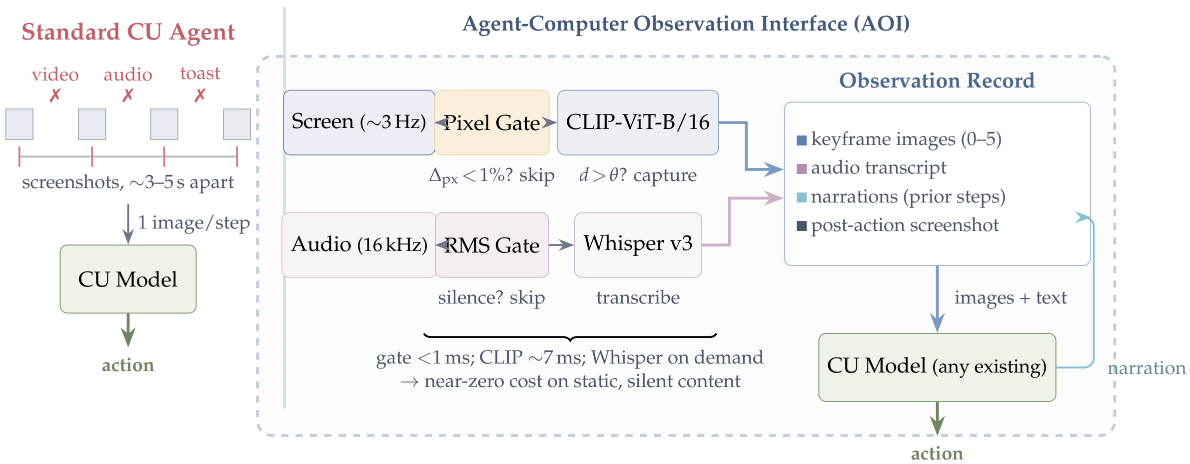


Figure 2. Standard CU agents vs. AOI-equipped agents. *Left:* Current agents observe through periodic screenshots (~3–5 s apart) and are blind and deaf between observations—missing video, audio, and transient UI events (✗). *Right:* The AOI continuously monitors screen and audio streams through fast gates (<1 ms) that skip processing for static, silent content. When gates fire, keyframe extraction (shown here with optional CLIP filtering; see Section 6.2) and Whisper ASR produce images and text for the observation record. Visual narrations feed back as persistent text memory. The AOI wraps any existing CU model with zero retraining.

Algorithm 1 Two-stage adaptive keyframe extraction (CLIP variant). This algorithm is shown for completeness; the selection-method ablation in Section 6.2 finds it statistically indistinguishable from the simpler pixel-only gate (Stage 1) and uniform / random sampling, so the default selector in every main result is Stage 1 alone.

Require: Screen sample x_t, anchor embedding e_{anchor}, pixel threshold α, CLIP threshold θ

$\Delta_{\text{px}} \leftarrow \text{PixelChangeRatio}(x_t, x_{\text{prev}})$
if $\Delta_{\text{px}} < \alpha$ **then**
    return SKIP {Stage 1: no pixel change}
end if
$e_t \leftarrow \text{CLIP}(x_t)$
$d \leftarrow 1 - \cos(e_t, e_{\text{anchor}})$
if $d > \theta$ **then**
    $e_{\text{anchor}} \leftarrow e_t$ {Re-anchor}
    return CAPTUREKEYFRAME(x_t, t)
end if
return SKIP {Stage 2: below semantic threshold}

non-speech audio events.

Volume gate. In most computer use—file editing, form filling, silent web browsing—there is no audio. Computing RMS energy of the audio buffer is sub-millisecond. If below the silence threshold, the audio model call is skipped entirely. Cost: ~0 ms for the majority of steps.

When audio is present, we use Whisper large-v3 [Radford et al., 2023] for speech transcription, operating on 16 kHz mono audio captured from the system audio output via PulseAudio virtual devices. Our implementation transcribes *speech* only; non-speech audio events (notification chimes, error beeps) trip the volume gate but produce no useful transcript, so the audio channel's contribution throughout this paper is spoken content—a multimodal audio model would be needed to cover the full audio scene (Section 10). The audio buffer spans the full inter-step interval including the post-action buffer, so speech that begins just after an action (e.g. a spoken confirmation) is still captured.

Overlapping windows. Agent step boundaries are determined by model inference latency, not by speech content. A typical 3.5-second audio chunk captures ∼8–9 words at normal speaking rate, almost always falling mid-sentence. To resolve boundary artifacts, the audio buffer includes ∼3.5 seconds of overlap from the previous interval, providing acoustic continuity across step boundaries.

3.4 Visual Narration for Long-Term Context

CU models retain only the most recent images in context. Keyframes from earlier steps are pruned. For tasks spanning many steps, the agent loses all visual memory of earlier content. Audio transcriptions persist naturally as text in the trajectory, but visual observations have no analogous persistence.

At each step, the CU model outputs both an action and a brief visual narration—a text description of what is visually new in the current keyframes. This narration is generated in the same inference call that produces the action—adding no extra inference call, only a small number of output tokens—and persists indefinitely in the trajectory, even after the corresponding images are pruned. Following the caption-then-reason principle of LLoVi [Zhang et al., 2023], narrations are generated by the CU model itself (no separate captioner) and are task-relevant rather than generic.

3.5 Observation Record

At each step, the CU model receives a structured document combining text context from recent prior steps (audio transcriptions, visual narrations, actions taken) and raw observations from the current interval (new audio transcription, any captured keyframe images, and the post-action screenshot). For static, silent tasks, the raw-observation part of this document reduces to just the post-action screenshot—no keyframes, no audio—but the surrounding structured format (DOM element list, prior-step text trajectory) is still present, so it is not identical to a raw-screenshot loop; Section 6.1 shows this format alone gives a small no-degradation gain on purely static work. Figure 3 shows a concrete example of the document format; Figure 4 grounds the pipeline in real pixels and verbatim narrations from a recorded benchmark run.

3.6 Implementation

The AOI is a modular Python layer (∼2,600 lines) that wraps any CU model. Screen and audio are captured through a headless browser and virtual audio devices; the keyframe encoder runs on GPU and speech transcription as a separate CPU service to avoid contention with the model. Cloud models are called through their native APIs and local models served on a single GPU. Full software, hardware, and hyperparameter details are in Appendix I.

4 DynaCU-Bench

Observation Record — Step N (Meeting task)

```
# CONTEXT -- text from prior steps (no images)
Step N-1 (t ≈ 18.5-22.0 s):
AUDIO: "Here's the team.  As you can see we
have strong cross-functional coverage."
VISUAL: "Meeting slide shows Project Team
with 6 members listed in a grid."
ACTION: wait()
# NEW -- current interval observations
Step N (t ≈ 22.0-25.5 s):
AUDIO: "And I want to mention, we've moved
the launch date to April 28th.
Please update your calendars."
[IMAGE: post-action screenshot]
# TASK
Read the meeting slides and listen to the
discussion.  What is the new launch date?
Enter it in the Launch Date field.
```

Figure 3. Example observation record sent to the CU model at step N of a meeting task. The **context** section provides text from prior steps (audio transcriptions and visual narrations persist after images are pruned). The **new** section contains the current audio transcription and the post-action screenshot (image). In this case, no keyframes were captured (the slide did not change), so only the screenshot is included as an image. The task instruction is appended at the end.

4.1 Design Principles

Existing CU benchmarks [Xie et al., 2024, Zhou et al., 2023, He et al., 2024] consist almost entirely of tasks solvable from static screenshots. We introduce **DynaCU-Bench**, a benchmark of 100 browser-based tasks that specifically require dynamic visual and/or audio perception—tasks that are unsolvable from static screenshots alone.

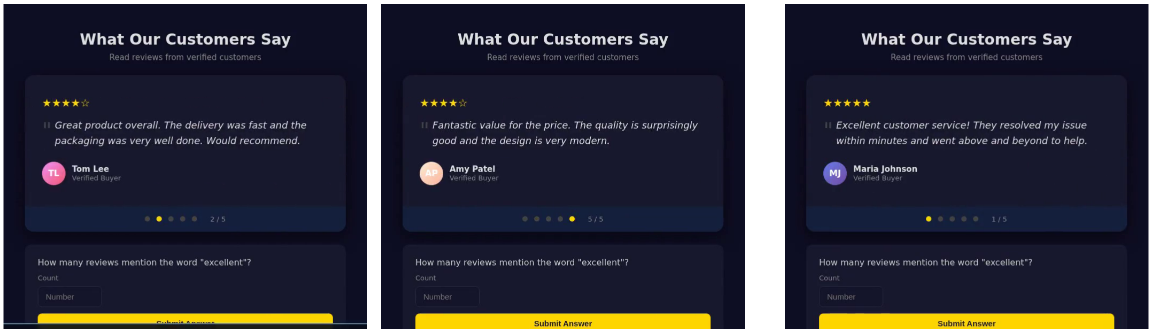
Design principles: (i) tasks must *require* temporal visual or audio perception; (ii) tasks represent realistic computer-use scenarios, not artificial constructs; (iii) tasks execute in a reproducible headless browser environment; (iv) tasks cover all four dynamic content categories from Section 2; (v) tasks include difficulty stratification (3 easy, 4 medium, 3 hard per category).

4.2 Task Categories

DynaCU-Bench spans 10 task families across three capability axes—audio perception (AUD), visual-temporal perception (VIS), and real-time interaction (INT). Throughout the paper we name each family descriptively (Table 1) rather than by an abstract letter, and use these names consistently in every table and figure. Each family has 10 tasks (3 easy, 4 medium, 3 hard); representative examples appear in Appendix F.

4.3 Evaluation Protocol

Each task runs in a headless Chromium browser with virtual audio devices for playback and microphone injection; spoken content is rendered by high-quality text-to-speech. Most tasks (93) are scored by a deterministic check on the resulting page state; the rest combine that check with an LLM quality rubric (6) or use an LLM judge alone (1). A run ends at 15 agent steps or a per-task wall-clock budget (the stimulus duration plus a fixed margin and any AOI



step 1: testimonial 2/5 (Tom Lee) step 2: testimonial 5/5 (Amy Patel) step 3: testimonial 1/5 (Maria Johnson)

Visual narrations accumulated in the trajectory (verbatim from the evaluation log)

`Step 1 (0 keyframes): "The carousel shows testimonial 2/5 from Tom Lee with a 4-star rating about fast delivery and good packaging, with no mention of 'excellent'." ACTION: click(567, 400)`

`Step 2 (1 keyframe): "The carousel now shows testimonial 5/5 from Amy Patel with a 4-star rating about fantastic value, quality, and modern design -- no mention of 'excellent'." ACTION: click(567, 401)`

`Step 3 (2 keyframes): "The carousel has cycled back to testimonial 1/5 from Maria Johnson with 'Excellent customer service!' -- and the transition keyframe also shows a Sarah Robinson testimonial partially visible..." ACTION: fill("2")` ✓

Figure 4. What the agent actually sees: real frames and narrations from a recorded benchmark run (Carousel task, "How many reviews mention the word 'excellent'?", Claude Sonnet 4.6 + AOI full—the run reported in Table 2; 3 steps, passed). The testimonial carousel rotates every few seconds, so no single screenshot can show more than one of the five reviews. *Top:* screen states observed across the three agent steps (frames from the run recording, recorder status bar cropped). *Bottom:* the visual narrations the CU model generated at each step, quoted verbatim from the evaluation log. The narrations persist as text in the trajectory, letting the agent accumulate evidence across carousel states and answer "2" at step 3.

overhead), whichever comes first. These budgets bind only for the slowest models (e.g. Grok-4 at ~80–180 s/call; Section 7).

5 Evaluation

5.1 Setup

CU models. We evaluate six models spanning the capability and scale spectrum: (i) Claude Sonnet 4.6 [Anthropic, 2024a] (closed-source); (ii) GPT-5.4 [OpenAI, 2026] (closed-source; the latest GPT-5 series tool-calling-capable vision model exposed by OpenAI's chat-completions API at evaluation time); (iii) Gemini 2.5 Flash [Google DeepMind, 2025] (closed-source); (iv) Grok-4 [xAI, 2026] (closed-source, xAI; reported in Section 7 with attention to the inference-latency confound); (v) EvoCUA-32B [Meituan, 2026] (open-source, 32B; weights and inference recipe released through Meituan's GitHub); (vi) Fara-7B [Awadallah et al., 2025] (open-source, 7B). Two further open-source models—Qwen3-VL-235B-A22B-Instruct and Qwen3-VL-30B-A3B-Instruct [Qwen Team, 2025], accessed through OpenRouter—are evaluated on the same standard vs. AOI full contrast as a replication (Table 3). We use each model's vendor-recommended decoding settings (temperature, top-p) unchanged. We do not rely on any vendor-reported

Table 1. DynaCU-Bench task families. Axes: AUD = audio perception, VIS = visual-temporal perception, INT = real-time interaction. Each family contains 3 easy, 4 medium, and 3 hard tasks.

Family	Axes	Description
Podcast	AUD	Listen to spoken narration; extract facts or answer questions
Meeting	AUD+VIS+INT	Follow a meeting with slides and speech; take notes or act
Screencast	VIS	Watch terminal/IDE recordings with auto-advancing frames
Carousel	VIS	Perceive CSS transitions, rotating content, product carousels
Dashboard	VIS	Monitor live-updating dashboards
Transient	VIS	Respond to toasts and auto-dismissing banners
Phone	AUD+INT	Listen to a synthesized voice call and respond
Interview	AUD+INT	Answer spoken questions; complete interviews
Collab	VIS+INT	Handle remote edits in collaborative documents
Games	VIS+INT	Play interactive games requiring temporal attention

Table 2. DynaCU-Bench results: task success rate (%) on the 100 dynamic tasks. Brackets show 95% Wilson confidence intervals. **Bold** indicates the best result per model. Δ is the absolute gain over the standard baseline; p-values are paired McNemar's exact mid-p tests. "Claude 4.6" abbreviates Claude Sonnet 4.6 throughout. Grok-4 is the slowest model at inference (∼80–180 s/call), which holds its absolute scores down even when AOI unblocks perception; see Section 7.

Configuration	Claude 4.6	GPT-5.4	Gemini 2.5	Grok-4	EvoCUA-32B	Fara-7B
Standard (screenshot)	38 [29.1, 47.8]	37 [28.2, 46.8]	21 [14.2, 30.0]	4 [1.6, 9.8]	18 [11.7, 26.7]	17 [10.9, 25.5]
AOI (full)	**82** [73.3, 88.3]	**57** [47.2, 66.3]	**69** [59.4, 77.2]	**25** [17.5, 34.4]	**55** [45.2, 64.4]	**34** [25.5, 43.7]
Δ (full − std)	+44	+20	+48	+21	+37	+17
p (McNemar)	1.3×10^{-10}	8.8×10^{-5}	2.9×10^{-12}	3.0×10^{-6}	3.0×10^{-9}	4.9×10^{-4}

OSWorld score for any claim in this paper; we report only the numbers measured under our own DynaCU-Bench harness.

Observation configurations. The primary contrast is **Standard** (one screenshot per step, no audio—the current paradigm) vs. **AOI full** (inter-step keyframes + audio transcription + visual narration), evaluated with all six models. The remaining modes are ablations and diagnostic controls run on Claude Sonnet 4.6 unless noted; a glossary of every mode used in the paper is in Appendix H.

5.2 Main Results

Table 2 and Figure 5 present the main results. The AOI produces large, consistent gains across all six models, all highly significant by paired McNemar's exact test ($p < 10^{-3}$ for every model; $p < 10^{-9}$ for Claude and Gemini, $p < 10^{-8}$ for EvoCUA). *Note on statistical power:* Each configuration is evaluated once on 100 binary-outcome tasks (no repeated trials). We

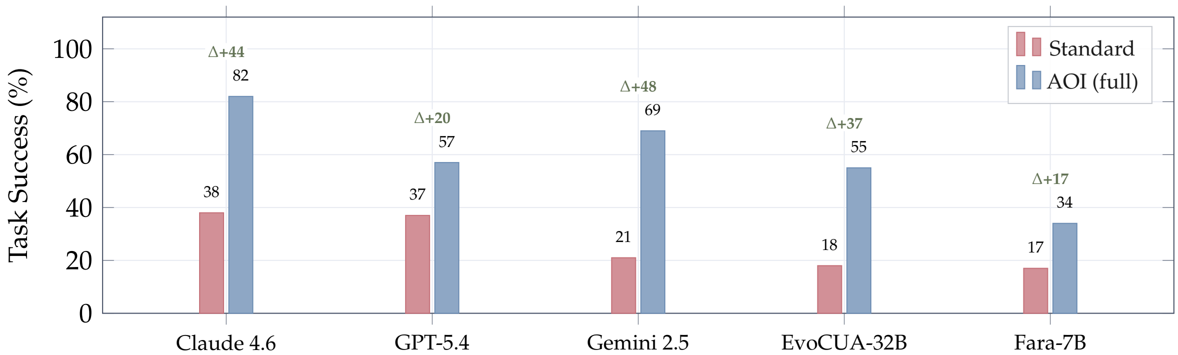


Figure 5. Main results on DynaCU-Bench (100 tasks). The AOI produces large gains across CU models without any retraining. Green numbers show absolute improvement in percentage points. Gemini 2.5 Flash shows the largest absolute gain (+48 pp) and relative gain (3.3×), while Claude achieves the highest absolute score (82%, seed 1; 3-seed mean 78.7%). Five of the six models tested are plotted here; Grok-4 (standard 4, AOI full 25, Δ+21 pp) is omitted from the chart for legibility and reported separately in Section 7. Exact (asymmetric) 95% Wilson confidence intervals on every bar are tabulated in Table 2 and used for all reported p-values.

use paired McNemar's exact mid-p tests on the discordant tasks to assess significance and report 95% Wilson confidence intervals on the marginal success rates. Pairwise differences between selection methods (e.g., 56% vs. 58%) are not significant ($p > 0.5$ for every pair; Appendix C); the large between-tier gaps are. The same single-trial caveat applies to every per-family, ablation, and per-difficulty cell in the paper: the three-seed sweep below puts the decoding-noise band at $\sim$3 pp on a 100-task total, so small differences (a few points on a 100-task total, $\pm$1–2 on a per-family /10) should be read as within that band unless a paired test says otherwise.

Claude Sonnet 4.6 is the strongest model, improving from 38% to 82% (+44 pp); Section 6 decomposes how the components combine to reach it.

GPT-5.4 improves from 37% to 57% (+20 pp). The gain is consistent with Claude's but the absolute ceiling is lower, suggesting GPT-5.4's CU capabilities are the binding constraint once perception is adequate.

Gemini 2.5 Flash scores 21% on the standard baseline despite being a video-native model with built-in streaming capabilities. This confirms that video-native architectures do not automatically solve the observation problem—the standard CU agent loop still constrains Gemini to periodic screenshots. With the AOI, Gemini improves to 69% (+48 pp)—the largest absolute gain of any model tested. Section 6 (Table 11) decomposes this gain into +25 pp from the AOI's structured prompt format and +23 pp from new perception: a video-native model benefits from *both* better presentation of existing context and explicit between-step perception.

EvoCUA-32B improves from 18% to 55% (3× improvement, +37 pp). This open-source 32B model, which barely functions on dynamic tasks without the AOI, becomes competitive with GPT-5.4's AOI-equipped performance.

Fara-7B improves from 17% to 34% (+17 pp), doubling its score. As the smallest model (7B parameters), it is most constrained by reasoning capacity, but still benefits substantially from improved perception.

Figure 6 makes the mechanism behind these gains concrete with two side-by-side trajectories: on a meeting task the standard agent waits out its entire step budget because it cannot

Table 3. Additional open-source replication on the full 100-task DynaCU-Bench: Qwen3-VL family via OpenRouter, evaluated with the same harness as the six models in Table 2. p-values are paired McNemar exact tests on the discordant tasks (b/c = success only under standard / only under AOI). Both gains fall inside the +17 to +48 pp band of the main table.

Model	Standard	AOI full	Δ (pp)	b/c	p (McNemar)
Qwen3-VL-235B-A22B-Instruct	22	**64**	+42	4/46	4.5×10^{-10}
Qwen3-VL-30B-A3B-Instruct	18	**42**	+24	8/32	1.8×10^{-4}

hear the spoken answer, while the AOI agent transcribes it and acts in two steps; on a video task the AOI agent's narration captures the answer in a single step.

Additional open-source replication (Qwen3-VL family). To probe the model-agnostic claim further, we run the same standard vs. AOI full contrast on two more open-source vision-language models via OpenRouter (provider: DeepInfra): **Qwen3-VL-235B-A22B-Instruct** [Qwen Team, 2025], the largest open-source VLM MoE available at evaluation time, and the smaller **Qwen3-VL-30B-A3B-Instruct**. Both land inside the +17 to +48 pp band of Table 2 (Table 3): +42 pp on the 235B model (22 $\rightarrow$ 64, McNemar $p = 4.5 \times 10^{-10}$) and +24 pp on the 30B model (18 $\rightarrow$ 42, $p = 1.8 \times 10^{-4}$). The 235B model's AOI full score (64) exceeds GPT-5.4's (57), and within the Qwen family the AOI gain grows with scale (+24 vs. +42 pp), consistent with the reasoning-capacity bottleneck observed on Fara-7B. (Two further candidates, UI-TARS-1.5-7B and GLM-4.5V, were unavailable through our OpenRouter provider list at evaluation time.)

Variance. Each cell above is one trial of 100 binary tasks at the model's default sampling temperature, folding decoding noise into the score. Re-running the headline Claude + AOI full configuration for two more seeds gives 82/78/76 (mean 78.7%, std 3.1 pp), so the 82% headline sits at the upper edge of a $\sim$3 pp band. This decoding band is separate from the McNemar question: McNemar asks whether two configurations differ given a fixed run, the seed sweep asks how far a single run's absolute score moves under fresh sampling.

5.3 Per-Family Analysis

Figure 7 maps the AOI gain across families and models (full per-cell scores in Appendix A). Three patterns stand out. The *audio* families (Podcast, Phone, Meeting) move from near-impossible to near-solved—these tasks cannot be done from screenshots at all, so audio perception is the whole story. The *visual-temporal* families (Screencast, Carousel, Transient) improve consistently; Dashboard gains least because a periodic value is usually visible in a single screenshot already. GAMES is the one family that regresses across models: it is bound by action latency, not perception, so the AOI's per-step observation work pushes the agent past the reaction window it would otherwise meet—the AOI targets human-paced use, not real-time play. Two model-level reads also emerge: the open-source EvoCUA-32B gains most on audio (weak visual grounding leaves more headroom there), while every model's Dashboard and Games cells stay flat-to-negative, isolating the gains to genuine perceptual slack.

5.4 Difficulty Breakdown

The AOI lifts every difficulty tier (full breakdown in Appendix A, Table 7): for Claude, easy rises 57% $\rightarrow$ 93%, medium 38% $\rightarrow$ 85%, hard 20% $\rightarrow$ 67%. Hard tasks remain hardest because

Meeting task — "What is the new launch date?"

Standard (screenshot-only) — FAIL

15 steps, 51.2 s

1. `wait()` — sees static slide
2. `wait()` — same slide
3. `wait()` — same slide
4. `wait()` — same slide
5. `wait()` — same slide

⋮

15. `wait()` × step budget exhausted

Agent never hears the spoken date.
Waits all 15 steps without ever acting.

AOI (full) — PASS

2 steps, 26.2 s

1. `wait()`
   Audio: "Here's the team. As you can see we have strong cross-functional coverage."
   Narration: "Meeting slide shows Project Team with 6 members."
2. `fill("April 28th")` ✓
   Audio: "We've moved the launch date to April 28th. Please update."
   Narration: "Audio announced the new launch date."

Agent hears "April 28th" and fills the form immediately.

Screencast task — "What was the first command in the terminal recording?"

Standard — PASS (6 steps)

1. `wait()` — recording playing
2. `wait()` — missed first command
3. `wait()` — scrolling through
4. `wait()`
5. `wait()`
6. `fill(...)` ✓ lucky timing

AOI — PASS (1 step)

1. `fill("git clone ...")` ✓
   Narration: "The terminal recording shows the first command was git clone. . . "

Narration captures the command from the recording immediately.

Figure 6. Trajectory comparison on two representative tasks (*illustrative; cherry-picked to expose the mechanism*). **Top:** Meeting task where the launch date is spoken aloud. The standard agent cannot hear it and exhausts its step budget on 15 futile `wait()` steps. The AOI agent transcribes the audio, hears "April 28th," and acts in 2 steps. **Bottom:** Screencast task where the answer appears in a terminal recording. The standard agent needs 6 steps with lucky timing; the AOI agent's narration captures the content in 1 step.

they need perception *and* reasoning—and on the smallest model (Fara-7B) the medium/hard gains stay small while easy nearly doubles, locating its ceiling at reasoning capacity rather than perception.

5.5 Efficiency

Better perception means fewer, better-aimed steps, which has a counter-intuitive cost consequence: *the AOI is cheaper than the screenshot baseline* on every cloud model despite its per-step observation work—token cost falls 15–50% (Claude and Gemini −50%, GPT-5.4 −15%) because the step count drops by about half and that outweighs the richer per-step input. Wall-clock time is the opposite trade: it *rises* for fast-inference models (the observation work exceeds the latency saved per skipped step) and falls only for the slowest one, where step reduction dominates. So the AOI is unambiguously preferred for batch or cost-sensitive use, and a per-model judgement call for latency-sensitive interactive use; the full instrumented breakdown is in Appendix A, Table 8.

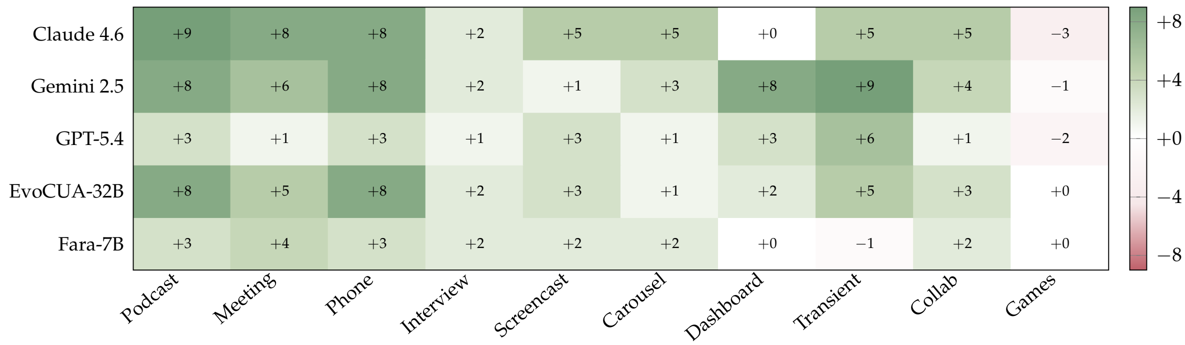


Figure 7. Per-family AOI gain (AOI full − Standard, in tasks per 10) across five models, families grouped by primary axis (audio, visual, interaction). Green = AOI helps, red = AOI hurts. Gains are positive almost everywhere and largest on the audio families (Podcast, Phone, Meeting), which are impossible from screenshots alone; the lone consistent regression is GAMES, where action latency—not perception—is the bottleneck. Absolute per-cell scores are in Appendix A.

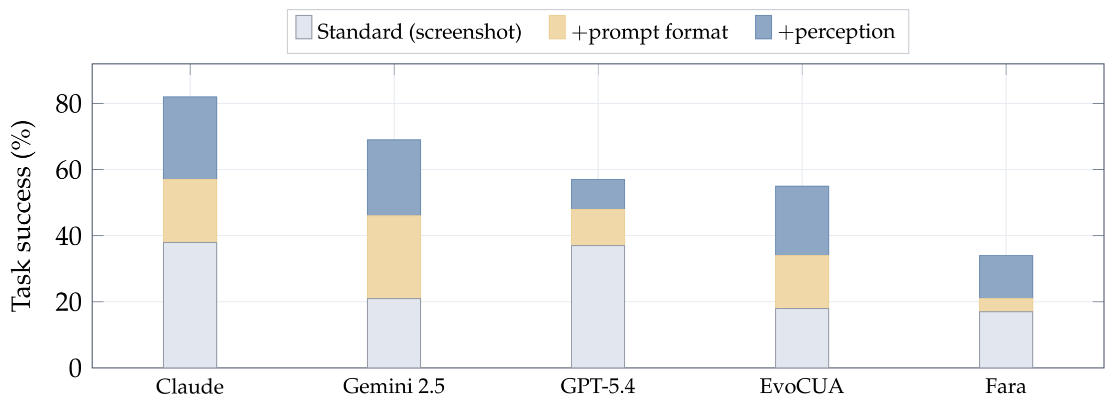


Figure 8. The AOI gain splits into a prompt-format and a perception component—both positive for every model. Bars stack the raw-screenshot baseline, the gain from the AOI's structured observation record alone (no keyframes/audio), and the further gain from perception (keyframes + audio + narration). The worst-case view that the gain is "just reformatting" is ruled out; the split is model-specific (Claude perception-leaning, Gemini 2.5 prompt-leaning, the small Fara-7B unable to exploit a richer prompt). Per-model numbers in Appendix B.

6 Decomposing the Gain

Where does the AOI gain come from? We answer in two layers: first separating a prompt-format effect from a genuine perception effect (Section 6.1), then opening up the perception effect channel by channel (Sections 6.2–6.4).

6.1 Prompt Format vs. Perception

Before crediting the gains to perception, we rule out the worst-case confound—that the AOI helps only by reformatting the prompt. Two checks settle it.

First, on **Static-50**—50 purely static, silent HTML tasks (forms, tables, calculations) whose rendered page never changes—the AOI's gates fire almost never (7 keyframes across all 50 tasks, no audio) yet it scores 50/50 against the screenshot baseline's 43/50. A perfect score on a workload with near-zero perception extraction is a prompt-format effect, not a hidden

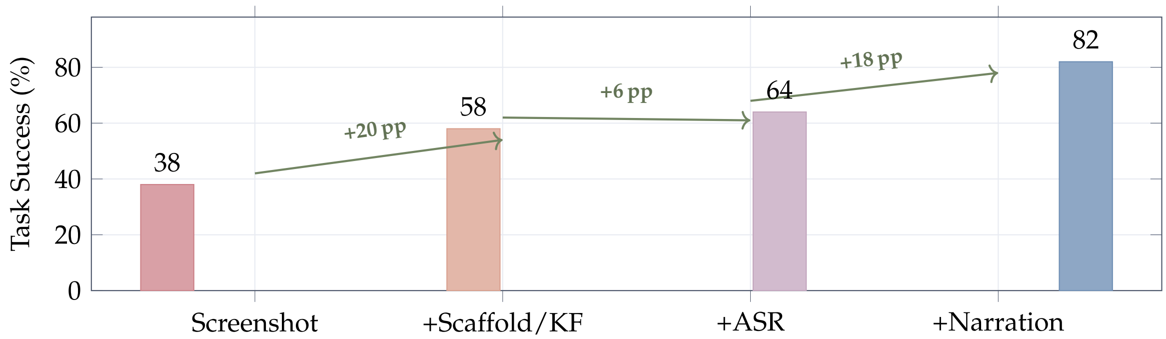


Figure 9. Progressive contribution of each AOI component (Claude Sonnet 4.6). The first step (+20 pp) *bundles* inter-step keyframes with the structured prompt scaffold that bare screenshots lack; the scaffold carries +19 of it (Section 6.1), the keyframe images +1 pp as raw input but +10 pp once narration is present (Figure 10b). ASR adds +6 pp (directional, $p = 0.18$); visual narration adds +18 pp, the largest single content component. Per-tier significance tests are in Appendix C.

perception gain, and it confirms the AOI does not degrade static work (Appendix B).

Second, on the dynamic benchmark we separate the two components directly: a control that supplies the AOI's structured observation record but disables all keyframe and audio extraction isolates the prompt-format effect, and the remainder is perception. Figure 8 shows both are positive for every model, so neither the "all reformatting" nor the "all perception" reading survives. The split is model-specific: Claude is perception-leaning, Gemini 2.5 prompt-leaning (for a video-native model, much of the gain is simply a better way to present context it already had), and the small Fara-7B cannot exploit the richer prompt at all. Within the prompt-format component a DOM-element list is the single largest lever (Appendix B). The rest of this section opens up the *perception* component—what each channel (keyframe selection, keyframe images, audio, narration) contributes, and how.

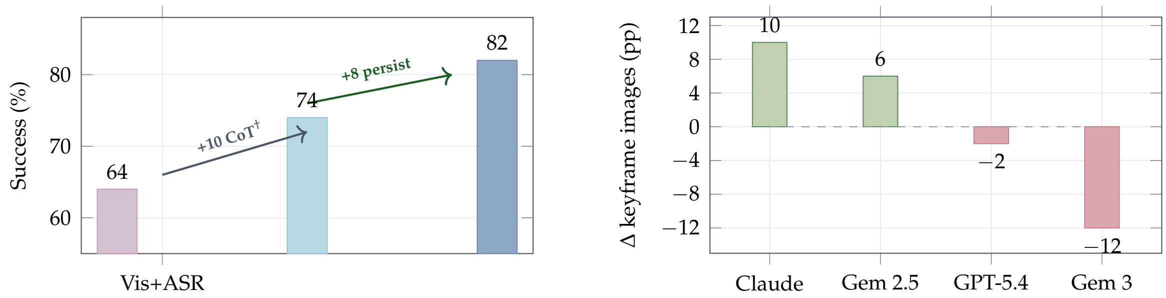
6.2 Keyframe Selection Is Immaterial; Images Pay Off Through Narration

Figure 9 walks Claude up the component ladder; two facts about the keyframe channel stand out.

Selection is immaterial. Five visual-only selection strategies—uniform 1 and 3 FPS, pixel-difference, random, and the two-stage CLIP filter of Algorithm 1—all land in a 56–58% band, statistically indistinguishable (every pairwise McNemar $p > 0.5$; Appendix C). Any inter-step frame breaks the agent's blindness; *how* it is chosen does not matter—a result that also holds on an open-source model (Qwen3-VL-32B, $p > 0.4$ for every pair; Appendix C) and is insensitive to the CLIP threshold across a $15\times$ range (Appendix G). No learned or semantic frame selector is needed.

The images pay off only once narrated. The +20 pp first step mostly turns on the structured scaffold that bare screenshots lack (+19 pp on its own; Section 6.1), not the keyframe images: holding the scaffold fixed, the images alone add just +1 pp. But once the model narrates what it sees, the same images are worth +10 pp (Figure 10b), and that lift lands precisely on the families where content changes *between* steps (Transient, Carousel), not where one screenshot per step already suffices. The keyframe channel is thus synergistic with narration—its model-dependence is the subject of Section 6.4.

Audio and narration. Speech transcription lifts 58% $\rightarrow$ 64% (directional at N=100, $p = 0.18$;



(a) Narration mechanism (Claude).

(b) Keyframe-image value, in context.

Figure 10. What narration and keyframes contribute. (a) Visual narration lifts Claude from 64% (visual+ASR, no narration) to 82%: discarding the narration text after it is generated keeps 74%, splitting the $+18$ pp into $\sim$+10 pp inference-time articulation (†directional, p=0.12) and $+8$ pp from persisting it as memory (p=0.039). **(b)** The marginal value of the inter-step keyframe *images*, measured with audio and narration already present, is model-specific: $+10/+6$ pp on Claude/Gemini 2.5 (realised only through narration—just $+1$ pp as raw input), neutral on GPT-5.4, and a -12 pp *penalty* on Gemini 3 (image-token dilution, Section 7).

confirmed at scale by the audio-family gains of Figure 7 and the Gemini 3 decomposition of Section 7), concentrated on the Phone and Interview families. Visual narration is the largest single content component, $+18$ pp (64% $\to$ 82%, $p = 5.3 \times 10^{-4}$); whether it works as persistent memory or as inference-time articulation is the question of the next subsection.

6.3 Narration: Articulation or Persistent Memory?

Narration does two things at once: it makes the model *articulate* the new visual content as it reasons, and it *persists* that text so it survives keyframe pruning. To separate them we run a *narration-discarded* control—the model narrates exactly as in AOI full (preserving any inference-time benefit), but the narration text is dropped before it can be reused. This lands at 74/100, between visual+ASR (64) and full (82) (Figure 10a), splitting the $+18$ pp into a directional $+10$ pp from articulation (64 $\to$ 74, $p = 0.12$, underpowered—$p < 0.05$ would need $N \sim 300$) and a significant $+8$ pp from persistence (74 $\to$ 82, $p = 0.039$).

What is the persistence effect doing? An external-judge audit of the ten tasks that full wins but narration-discarded loses finds the prior-step narrations literally contain the load-bearing fact in only two—both genuinely multi-step (breakpoints recorded across video frames; page titles recorded as a sequence advanced). In the other eight the needed information is present *within* the winning step itself, so persistence there is not supplying a verbatim answer but steering earlier steps onto a better trajectory (three first-step wins are pure decoding variance). Narration's primary mechanism is therefore articulation; persistence matters most where information genuinely spans steps.

6.4 The Keyframe Channel Is the Most Model-Dependent

Measured in the deployed configuration—audio and narration already on—the marginal value of the inter-step keyframe *images* swings from $+10$ pp on Claude and $+6$ pp on Gemini 2.5 (both realised only through narration; the same images add $+1$ pp without it), through a neutral -2 pp on GPT-5.4, to a -12 pp *penalty* on Gemini 3 (Figure 10b). The positive lift concentrates on the families where frames carry novel between-step content (Transient, Carousel) and is

Table 4. Later-released models (100 dynamic tasks each, same harness). Gemini 3 checks whether the Gemini 2.5 +48 pp gain replicates; the two Grok rows control for the inference-latency floor.

Model	Standard	AOI full	Δ (pp)	p (McNemar)
Gemini 3 Flash	36	45	+9	0.18
Grok-4.3	25	65	+40	8.2×10^{-10}
Grok-4-fast-reasoning	19	47	+28	4.9×10^{-6}

near-zero on audio and static families, pinning it to narration rather than audio. The lesson generalises into the operative rule of Section 7: observation delivered as *text*—transcripts and narration—is robustly positive on every model, whereas the raw image stream must be treated as a per-model toggle. The gates make that toggle cheap: only ∼28% of steps ever produce a keyframe and ∼14% activate audio, leaving ∼61% of steps fully idle (per-family activity in Appendix D). A cross-engine check—audio re-rendered with a different speech synthesizer and real terminal-recording screencasts—confirms the audio pipeline does not silently fail under an unfamiliar acoustic profile (Appendix J).

7 Model Case Studies: Later-Released Models

Do the AOI's gains hold up on models released after the main runs? We ran three more 100-task evaluations with the same harness (Table 4). Together with the original Grok-4, they form two instructive case studies: a family where the binding constraint is inference *latency* rather than perception (Grok), and a model where one AOI component *flips sign* (Gemini 3).

Grok: latency, not perception, is the ceiling. The original Grok-4 scored only 4/100 standard and 25 with AOI (+21 pp, $p = 3 \times 10^{-6}$)—valid actions, but a single call takes 80–180 s, so the agent times out after one or two steps on harder tasks. The AOI helps where a few well-placed observations suffice but cannot unblock tasks needing many sequential actions, so +21 pp lower-bounds the true effect. The follow-ups confirm this: a lower-latency variant reaches 47 and the newer Grok-4.3 reaches 65, decomposing the ceiling into ∼+22 pp from removing latency and ∼+18 pp from base-model improvement.

Gemini 3: a component flips sign. Gemini 3 Flash nets only +9 pp ($p = 0.18$), down from Gemini 2.5's +48. This is not the model having internalised the AOI: a four-way decomposition (Figure 11; tables in Appendix E) shows it is a *cancellation*. Audio still helps (+12 pp, with +10 on the audio families alone—Gemini 3 is not audio-saturated), and the structured scaffold still helps on balance (+9 pp), but the inter-step keyframe images now *regress* −12 pp. Dropping just the keyframes recovers 57/100, a +21 pp gain—so the slack is real and the default bundle is simply mis-tuned. A causal probe isolates the mechanism (Appendix E): replacing the keyframes with pure-noise images of the same size reproduces the full penalty, and the cost does not scale with keyframe count or position—so it is *image-token dilution* (extra images degrade Gemini 3's action grounding regardless of content), not distraction by what the frames show. The actionable rule is simple: on this model, deliver observation as text and turn the keyframe-image stream off.

A diagnostic map of model slack. Read this way, the AOI's residual gain is a diagnostic of *where* a model has headroom, along richer axes than perception alone: prompt-handling (Gemini 2.5 leans prompt-format, $+25/+23$), perception (Claude leans perception, $+19/+25$), reasoning capacity (Fara-7B cannot exploit a richer prompt, $+4/+13$), latency (Grok), and

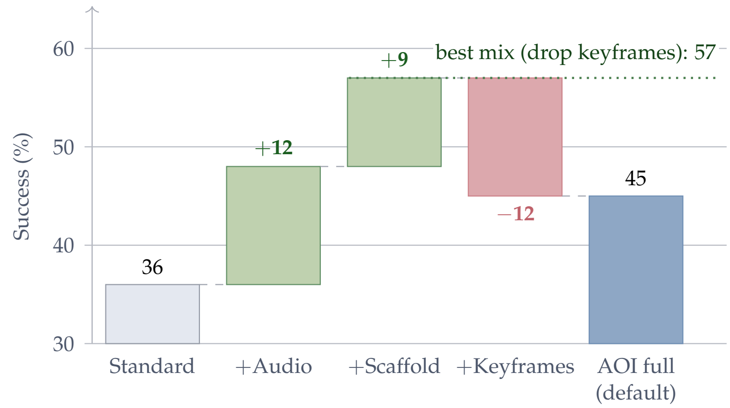


Figure 11. The AOI's small +9 pp net on Gemini 3 Flash is a three-way cancellation, not saturation. Audio (+12) and the structured scaffold (+9) help as on every other model, but the keyframe-image stream *regresses* −12 pp—so the default bundle nets only 36 → 45. Dropping just the keyframes recovers 57/100 (+21 pp over Standard), showing the slack is real and the bundle merely mis-tuned. Underlying per-mode and per-family tables are in Appendix E.

Table 5. Streaming multimodal baselines vs. the AOI on a 12-task audio subset (3 each from Podcast, Meeting, Phone, Interview). Each baseline is scored on the same task IDs as the AOI in Table 2. This is a deliberately favourable comparison—neither baseline is built for grounded computer use.

Configuration	Pod	Meet	Phone	Intv	Total / 12
OpenAI Realtime	0	0	0	3	3 (25%)
Gemini Live	0	0	0	0	0 (0%)
AOI full (Claude 4.6)	3	3	3	3	**12** (100%)

action-policy compatibility with the scaffold (Gemini 3). The interface is not a fixed bundle; its components must be selected per model.

8 Comparison with Streaming Multimodal Baselines

A natural alternative is a *streaming, end-to-end multimodal model* that bundles perception and reasoning in one trained model—today, Gemini Live [Google, 2025] and the OpenAI Realtime API [OpenAI, 2024]. Neither ships as a CU agent; each is a voice-first assistant with function-calling. We adapt both to DynaCU-Bench (one session per task, page audio and screenshots streamed in, returned tool calls executed in the browser), using the highest vision- and function-calling-capable tier of each, and score them on the same 12-task audio subset the AOI is scored on.

Both underperform the AOI by a wide margin (Table 5). OpenAI Realtime succeeds only on the simplest interview questions, answerable from a single audio chunk; on podcast and meeting tasks it waits indefinitely without extracting the spoken content. Gemini Live, optimised for audio-in/audio-out, returns few tool calls and tends to click non-actionable coordinates rather than commit a typed answer. A sanity check rules out a broken adapter: on five purely-visual tasks both baselines drive the browser and emit valid actions (5/5) but reach 0/5 task-correct—wrong targets, hallucinated text, premature commits—while AOI-equipped Claude passes 4/5 (Appendix K). The takeaway: a streaming voice model with function-calling is not a substitute for a perception layer that persists what it heard as text.

9 Related Work

Expanding the interface, not the model. Much agent progress comes from reshaping the agent–computer interface rather than the weights. On the *action* side, SWE-agent [Yang et al., 2024] redesigned the command set and feedback format, and agents adopt executable code actions [Wang et al., 2024a] and element-grounded actions [Deng et al., 2023]. The *observation* side is also well studied—but almost entirely as richer encodings of a *single static snapshot*: accessibility trees and HTML/DOM text surrogates [Zhou et al., 2023, Deng et al., 2023] (which serve as both observation and action representations), and set-of-mark element overlays that ground vision models on the screenshot [Yang et al., 2023, Zheng et al., 2024]. What none of these representations change is *when* the agent looks—one snapshot per action, and no audio. We expand observation along the orthogonal axes they leave out, time (continuous, between-step capture) and modality (sound); this is complementary to single-frame encodings and composes with them.

Computer-use agents. The CU agent landscape spans closed-source systems [Anthropic, 2024a, OpenAI, 2025, Google DeepMind, 2025, Andreux et al., 2025] and open-source models [Wang et al., 2025a,b, Awadallah et al., 2025, Ye et al., 2025, Song et al., 2025]. Agent S3 [Simular AI, 2025] demonstrates competitive open-source performance on OSWorld. A recent CU agent survey [Sager et al., 2025] catalogues many agents and datasets and identifies several open gaps, but does not identify the observation modality as one. All of these agents operate through the screenshot–reason–act loop; none addresses dynamic visual content or audio perception.

Real-time multimodal models. Gemini Live [Google, 2025] and the OpenAI Realtime API [OpenAI, 2024] process continuous audio/video/text streams in real time, bundling perception with reasoning into a single trained model. Our work is complementary: the AOI decouples perception from reasoning, so the same perception layer benefits any CU model without retraining. Section 8 compares the two paradigms head-to-head on the audio-focused DynaCU-Bench subset.

Video understanding for GUI. GUI-World [Chen et al., 2024], VideoWebArena [Jang et al., 2024], and CUA-Suite [Chen et al., 2025] identify the observation problem through benchmarking. MemGUI-Bench [Park et al., 2025] reveals memory gaps in GUI agents. These benchmarks identify the problem but do not propose solutions compatible with existing CU agents.

Keyframe selection. VideoTree [Wang et al., 2024b] and AKS [Tang et al., 2025] use CLIP-based frame selection for offline video QA. Apollo [Zohar et al., 2024] finds that FPS sampling outperforms uniform sampling and identifies optimal token budgets. All prior work targets offline video understanding; our contribution is real-time observation filtering for CU agents, where the two-stage design (sub-millisecond pixel gate followed by $\sim$7 ms CLIP only on changed frames) keeps the per-frame cost low enough to run continuously in the background between agent steps.

Adaptive middleware for GUI agents. Cradle [Tan et al., 2024] includes frame difference analysis for games—the closest predecessor to the AOI's perception layer—but uses pixel-level differencing without semantic filtering and has no audio processing. ScreenLLM [Jin et al., 2025] captures stateful screen schemas with keyframe extraction, complementing visual narration but without audio or adaptive observation. StreamAgent [Yang et al., 2025] and Dispider [Qian et al., 2025] separate perception from response generation for streaming video, paralleling the AOI's architecture, but neither addresses CU agent observation.

Structured agent interfaces. A complementary line of work bypasses the perception problem by having websites or applications expose structured interfaces directly to agents: the Model Context Protocol [Anthropic, 2024b] for tool/data exchange and declarative web specifications such as NLWeb [Microsoft, 2025] for site-side agent endpoints. These approaches are powerful where adoption exists, but require website cooperation; the AOI enables perception on arbitrary, unmodified web content (any HTML/audio rendered to a browser, including legacy or third-party pages with no agent endpoint).

10 Limitations

Latency and temporal resolution. The AOI extends perception but not decision speed: CU models still take 1–5 s per step, and the screen is sampled at $\sim$3 Hz, so sub-$\sim$300 ms events and real-time games fall outside its reach (the Games regression). It targets human-paced computer use, not fast interactive control.

Perception is not reasoning. Better inputs do not improve reasoning over those inputs; on the smallest model the gain concentrates on easy tasks and reasoning capacity becomes the binding constraint.

Audio is speech-only and narration can err. We transcribe speech but not non-speech events (chimes, beeps); a multimodal audio model would cover the full audio scene. Long-term visual memory also inherits the CU model's mistakes—a misread number persists in the trajectory.

External validity. Tasks run in a controlled browser with synthesized audio, so real-world content (live calls, native apps) may differ, and we do not yet report AOI numbers on existing dynamic-GUI benchmarks—those evaluate offline video QA or fixed-context video rather than a live observe–act loop with a separate audio channel, so wrapping them with the AOI requires re-instrumenting each harness. That re-instrumentation is the clearest external-validity test and the main item of future work.

Statistical power. Each cell is a single 100-task trial; we use paired McNemar tests for differences but the data does not support fine-grained ranking of configurations within a few points of each other.

11 Conclusion

Computer-use agents are blind between screenshots and deaf to audio because observation is tied to action: one screenshot per step. We argued the *observation* interface is a separable, underexplored design axis—the counterpart to the action interface SWE-agent surfaced—and introduced the AOI, a perception layer that is transparent on static work, adaptive on dynamic content, and compatible with any existing CU model without retraining. Across eight models it turns dynamic tasks that were near-impossible from screenshots into largely solvable ones, while reducing token cost.

Opening up that gain reframes CU perception in three ways: keyframe *selection* is immaterial and value emerges only once frames are narrated into persistent text; narration helps mainly by making the model articulate what it sees; and the interface is not a fixed bundle—one component even reverses sign on a later-generation model, so the components must be tuned per model rather than shipped as one configuration. We release the perception layer and benchmarks so future work can cleanly separate observation gains from model-side gains.

Acknowledgements

Pine Copilot, Claude Code, and Claude Opus 4.8 were used during this research. We thank BSQL Networking for hosting the NVIDIA RTX Pro 6000 GPU on which these experiments were run.

References

Mathieu Andreux et al. Surfer 2: The next generation of cross-platform computer use agents. *arXiv preprint arXiv:2510.19949*, 2025.

Anthropic. Claude computer use. `https://docs.anthropic.com/en/docs/agents-and-tools/computer-use`, 2024a.

Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024b.

Ahmed Awadallah et al. Fara-7B: An efficient agentic model for computer use. *arXiv preprint arXiv:2511.19663*, 2025.

Dongping Chen et al. GUI-World: A video benchmark and dataset for multimodal GUI-oriented understanding. *arXiv preprint arXiv:2406.10819*, 2024.

Liang Chen et al. CUA-Suite: 55 hours of real computer-use recordings for agent benchmarking. *arXiv preprint arXiv:2510.10142*, 2025.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Google. Live API (gemini live). `https://ai.google.dev/gemini-api/docs/live`, 2025.

Google DeepMind. Gemini 2.5 Computer Use. `https://deepmind.google/models/gemini/computer-use/`, 2025.

Hongliang He et al. WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.

Lawrence Jang et al. VideoWebArena: Evaluating long context multimodal agents with video understanding web tasks. *arXiv preprint arXiv:2410.19100*, 2024.

Yiqiao Jin et al. ScreenLLM: Stateful screen schema for efficient action understanding and prediction. *arXiv preprint arXiv:2503.20978*, 2025.

Meituan. EvoCUA: Evolving computer use agent. `https://github.com/meituan/EvoCUA`, 2026. Accessed 2026-05-14.

Microsoft. NLWeb: A conversational interface for websites. https://github.com/microsoft/NLWeb, 2025.

OpenAI. Realtime API: Speech-to-speech models with tool use. https://platform.openai.com/docs/guides/realtime, 2024.

OpenAI. Computer-using agent (CUA). https://openai.com/index/computer-using-agent/, 2025.

OpenAI. Introducing GPT-5.4. https://openai.com/index/introducing-gpt-5-4/, 2026.

Joonsuk Park et al. MemGUI-Bench: Benchmarking memory in GUI agents. *arXiv preprint arXiv:2509.12233*, 2025.

Rui Qian et al. Dispider: Enabling video LLMs with active real-time interaction via disentangled perception, decision, and reaction. *arXiv preprint arXiv:2501.03218*, 2025.

Qwen Team. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023. arXiv:2212.04356.

Alec Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. arXiv:2103.00020.

Pascal J. Sager et al. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *arXiv preprint arXiv:2501.16150*, 2025.

Simular AI. Agent S3: An open-source computer-use agent. https://github.com/simular-ai/Agent-S, 2025.

Linxin Song et al. CoAct-1: Computer-using agents with coding as actions. *arXiv preprint arXiv:2508.03923*, 2025.

Weihao Tan et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.

Xi Tang et al. Adaptive keyframe sampling for long video understanding. *arXiv preprint arXiv:2502.21271*, 2025.

Haoming Wang et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024a.

Xinyuan Wang et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025b.

Ziyang Wang et al. VideoTree: Adaptive tree-based video representation for LLM reasoning on long videos. *arXiv preprint arXiv:2405.19209*, 2024b.

xAI. Grok 4. https://x.ai/news/grok-4, 2026. Accessed 2026-05-14.

Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. NeurIPS 2024.

Haolin Yang et al. StreamAgent: Towards anticipatory agents for streaming video understanding. *arXiv preprint arXiv:2508.01875*, 2025.

Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023.

John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Jiabo Ye et al. Mobile-Agent-v3: Fundamental agents for GUI automation. *arXiv preprint arXiv:2508.15144*, 2025.

Alex L. Zhang et al. VideoGameBench: Can vision-language models complete popular video games? *arXiv preprint arXiv:2505.18134*, 2025.

Ce Zhang et al. A simple LLM framework for long-range video question-answering. *arXiv preprint arXiv:2312.17235*, 2023.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *International Conference on Machine Learning (ICML)*, 2024.

Shuyan Zhou et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. ICLR 2024.

Orr Zohar et al. Apollo: An exploration of video understanding in large multimodal models. *arXiv preprint arXiv:2412.10360*, 2024.

A Detailed Results

This appendix collects the per-cell numbers summarised by the figures in the body: per-family success for every model (Table 6, the absolute scores behind the gain heatmap, Figure 7), the difficulty breakdown (Table 7), efficiency and cost (Table 8), and the full per-family ablation sweep (Table 9).

Table 6. Per-family success (pass/10), Standard vs. AOI full, for the five models with full per-family breakdowns. Grok-4 is omitted (latency-limited; Section 7). These are the absolute scores behind Figure 7.

	Claude 4.6		GPT-5.4		Gemini 2.5		EvoCUA-32B		Fara-7B	
Family	Std	AOI	Std	AOI	Std	AOI	Std	AOI	Std	AOI
Podcast	0	**9**	0	3	0	**8**	0	**8**	0	3
Meeting	2	**10**	2	3	2	**8**	1	6	1	5
Screencast	4	**9**	5	**8**	5	6	3	6	3	5
Carousel	4	**9**	6	7	4	7	3	4	1	3
Dashboard	8	8	6	**9**	1	**9**	2	4	2	2
Transient	4	**9**	2	**8**	0	**9**	0	5	4	3
Phone	1	**9**	1	4	0	**8**	0	**8**	1	4
Interview	7	**9**	6	7	5	7	6	**8**	1	3
Collab	3	**8**	4	5	2	6	2	5	2	4
Games	5	2	5	3	2	1	1	1	2	2
Total	38	**82**	37	**57**	21	**69**	18	**55**	17	**34**

Table 7. Task success by difficulty (Standard → AOI full). Easy: 30 tasks; Medium: 40; Hard: 30.

	Standard			AOI Full		
Model	Easy	Med	Hard	Easy	Med	Hard
Claude 4.6	17/30	15/40	6/30	**28**/30	**34**/40	**20**/30
GPT-5.4	15/30	17/40	5/30	21/30	26/40	10/30
Gemini 2.5	10/30	9/40	2/30	24/30	33/40	12/30
EvoCUA-32B	6/30	8/40	4/30	20/30	26/40	9/30
Fara-7B	9/30	5/40	3/30	19/30	11/40	4/30

B Prompt-Format vs. Perception Decomposition

Numbers behind Figure 8. Table 10 is the Static-50 no-degradation check; Table 11 is the per-model prompt/perception split; and Table 12 shows that within the prompt-format effect a DOM-element list is the single largest lever.

C Statistical Tests and Component Decompositions

The per-tier significance tests behind Figure 9 (Table 13), the open-source replication of the selection-invariance finding (Table 14), the per-family narration decomposition behind Fig-

Table 8. Efficiency and cost on the 100-task benchmark (Grok-4 omitted; latency floor distorts the picture). Steps = mean steps/task; Time = mean wall-clock/task; Lat. = total inference latency/task; Obs = total observation processing/task; Tokens = input+output/task (estimated, same estimator for every row); $/100 tasks at May 2026 list prices (local models free). The AOI is cheaper in dollars on every cloud model because the step count drops faster than per-step input grows.

Configuration	Steps	Time (s)	Lat. (s)	Obs (s)	Tokens (k)	$/100
Claude Sonnet 4.6						
Standard	10.7	40.6	26.3	2.5	7.6	$2.72
AOI full	**4.8**	46.2	15.2	12.0	**3.8**	**$1.35**
GPT-5.4						
Standard	10.0	26.6	14.4	2.4	5.8	$3.23
AOI full	**7.6**	50.6	11.2	17.8	**5.0**	**$2.76**
Gemini 2.5 Flash						
Standard	11.5	55.3	40.4	2.6	7.6	$0.32
AOI full	**5.4**	61.9	20.6	20.6	**4.0**	**$0.16**
EvoCUA-32B (local)						
Standard	9.3	117.0	99.0	2.3	5.8	—
AOI full	**5.3**	113.9	72.0	22.6	**4.0**	—
Fara-7B (local)						
Standard	13.4	35.1	24.0	2.5	9.2	—
AOI full	**9.3**	90.7	17.0	31.6	**7.1**	—

Table 9. Complete per-family results for all 8 observation configurations tested with Claude Sonnet 4.6. Family abbreviations: Pod=Podcast, Meet=Meeting, Scr=Screencast, Car=Carousel, Dash=Dashboard, Tran=Transient, Phn=Phone, Int=Interview, Col=Collab, Gam=Games.

Mode	Pod	Meet	Scr	Car	Dash	Tran	Phn	Int	Col	Gam	Total
Standard	0	2	4	4	8	4	1	7	3	5	38
Uniform 1 FPS	0	4	10	8	9	8	1	8	6	4	58
Uniform 3 FPS	2	5	8	8	8	8	1	7	6	3	56
Pixel-diff	1	5	9	7	9	8	1	8	6	4	58
Random keyframes	1	6	8	6	9	8	1	8	6	3	56
AOI visual	1	5	9	7	8	9	1	8	6	4	58
AOI visual+ASR	1	4	10	9	9	9	4	9	6	3	64
AOI full	**9**	**10**	**9**	**9**	**8**	**9**	**9**	**9**	**8**	2	**82**

Table 10. Static-50 (50 purely static, silent tasks), Claude Sonnet 4.6. The gates suppress almost all extraction (7 keyframes total, no audio), yet the AOI scores 50/50—a prompt-format effect, not perception, and no degradation on static work.

Mode	Pass/50	Avg Steps	Total KF	Audio Seg.
Standard	43 (86%)	3.26	0	0
AOI full	**50** (100%)	**1.62**	7	0

ure 10a (Table 15), and the in-context keyframe-image decomposition behind Figure 10b (Table 16).

Table 11. Prompt-format vs. perception split on DynaCU-Bench (100 tasks). A structured-record control with no keyframes/audio isolates Δ_{prompt}; the remainder up to AOI full is Δ_{percept}. Both are positive for every model. (Gemini 3 Flash, where the split differs, is in Appendix E.)

Model	Standard	+format	AOI full	Δ_{prompt}	Δ_{percept}
Claude Sonnet 4.6	38	57	82	+19	+25
GPT-5.4	37	48	57	+11	+9
Gemini 2.5 Flash	21	46	69	+25	+23
EvoCUA-32B	18	34	55	+16	+21
Fara-7B	17	21	34	+4	+13

Table 12. Which prompt sub-component carries the load (Claude Sonnet 4.6, 100 tasks). Trajectory = prior-step action/text history and sectioning; PAGE EL = DOM-element list. The element list is the single largest lever (+30 pp added to the minimal prompt); the two are sub-additive.

Mode	Pass/100	Trajectory?	Element list?
Minimal (task + screenshot)	16	no	no
+trajectory (= Standard)	38	yes	no
+element list	46	no	yes
+both (= +format)	**57**	yes	yes

Table 13. Pairwise McNemar exact mid-p tests between successive ablation tiers (Claude Sonnet 4.6, 100 tasks). b/c are discordant counts (success only in the previous / only the current configuration). The selection-method group is statistically indistinguishable; the between-tier transitions are significant.

Comparison	Total	b	c	p
Standard → Uniform 1 FPS	38 / 58	3	23	8.8×10^{-5}
Uniform 1 FPS → Uniform 3 FPS	58 / 56	7	5	0.77
Uniform 3 FPS → Pixel-diff	56 / 58	4	6	0.75
Pixel-diff → Random keyframes	58 / 56	4	2	0.69
Random keyframes → CLIP adaptive	56 / 58	4	6	0.75
CLIP adaptive → AOI visual+ASR	58 / 64	4	10	0.18
AOI visual+ASR → AOI full	64 / 82	4	22	5.3×10^{-4}

Table 14. Selection-method ablation replicated on an open-source model (Qwen3-VL-32B), 50 visual-temporal tasks (Screencast, Carousel, Dashboard, Transient, Games). The three methods cluster with no pairwise significance ($p > 0.4$), mirroring Claude: the convergence is not model-specific.

Selection method	Pass / 50	Vs. adjacent
Uniform 1 FPS	26	—
Pixel-diff	27	vs. Uniform: $p = 0.69$
Random keyframes	27	vs. Pixel-diff: $p = 1.00$

D Per-Step Observation Activity

The gates suppress processing on the majority of steps: only ∼28% produce a keyframe and ∼14% activate audio (Table 17), and weighted by step count ∼61% of steps are fully idle (Figure 12). Audio-only families produce zero keyframes and visual-only families zero audio, confirming clean channel separation.

Table 15. Narration decomposition (Claude Sonnet 4.6, 100 tasks): visual+ASR (no narration) $\rightarrow$ narration-discarded $\rightarrow$ full. The discarded run isolates the inference-time articulation effect; the gap to full isolates persistence. Per-family rows are descriptive (single-trial); note Transient reads 9$\rightarrow$4$\rightarrow$9, a decoding fluctuation on a 10-task cell that the totals average out.

Configuration	Tot.	Pod	Meet	Scr	Car	Dash	Tran	Phn	Int	Col	Gam	Vs. adj.
Visual+ASR (no narr.)	64	1	4	10	9	9	9	4	9	6	3	—
Full, narration discarded	74	9	10	8	8	7	4	9	9	7	3	$p = 0.12$
Full (narration persisted)	**82**	9	10	9	9	8	**9**	9	9	8	2	$p = 0.039$

Table 16. Marginal value of the inter-step keyframe *images* measured in context (audio and narration present): scaffold+ASR+narration without keyframes vs. with them. Δ_{KF} ranges from +10 pp to a −12 pp penalty across models—the most model-dependent component. †GPT-5.4 here is measured through a single backend for both modes, so only its within-row Δ_{KF} is comparable to the absolute scores elsewhere.

Model	no keyframes	+keyframes	Δ_{KF} (pp)
Claude Sonnet 4.6	72	**82**	+10
Gemini 2.5 Flash	63	**69**	+6
GPT-5.4†	**72**	70	−2
Gemini 3 Flash	**57**	45	−12

Table 17. Per-step observation statistics for Claude Sonnet 4.6 + AOI full, by family. % KF = steps capturing $\geq$ 1 keyframe; KF/Step = mean keyframes/step; % Audio = steps with audio transcription.

Family	Steps	% KF	Tot. KF	KF/Step	% Audio
Podcast	35	8.6	3	0.09	42.9
Meeting	34	50.0	18	0.53	73.5
Screencast	39	28.2	12	0.31	2.6
Carousel	61	60.7	84	1.38	0.0
Dashboard	52	28.8	18	0.35	0.0
Transient	38	15.8	7	0.18	0.0
Phone	38	0.0	0	0.00	26.3
Interview	27	0.0	0	0.00	37.0
Collab	39	10.3	4	0.10	17.9
Games	118	36.4	65	0.55	0.0
All	**481**	**28.3**	**211**	**0.44**	**14.1**

E Gemini-3 Flash Decomposition

Tables underlying the Gemini 3 case study (Figure 11). Table 18 is the four-way decomposition behind the waterfall; Table 19 is the per-family AOI Δ (the shortfall vs. Gemini 2.5 sits entirely on Dashboard and Transient, where Gemini 3 alone regresses); Table 20 is the causal probe that pins the keyframe regression to image-token dilution.

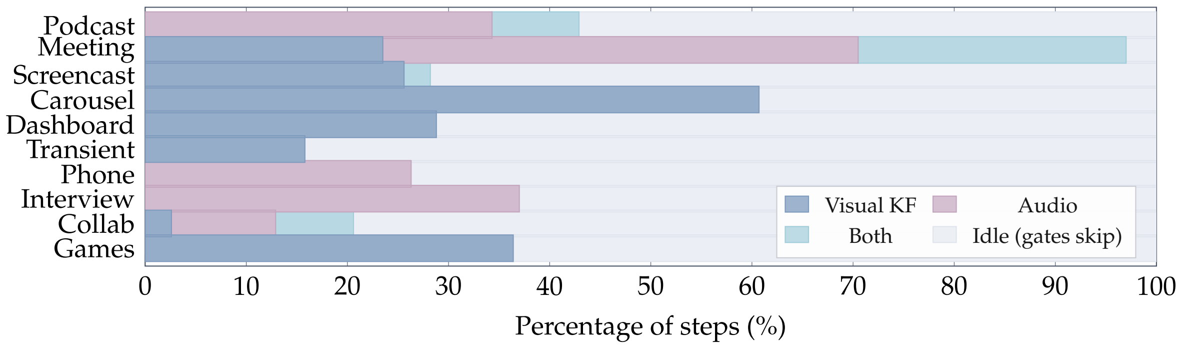


Figure 12. Observation activity per family. Percentage of steps in which each channel fires; gray is idle (both gates suppress). Carousel is highly visual; Phone and Interview are audio-only; Meeting uses both. ∼61% of steps are idle overall.

Table 18. Four-way decomposition of the Gemini 3 AOI residual. If the small net were internal saturation, the audio-plus-standard-prompt row would not exceed the full AOI. "Aud4" = the four audio families (Podcast, Meeting, Phone, Interview); "D+T" = Dashboard + Transient, where Gemini 3 regresses.

Mode	**Total**	**Aud4**	**D+T**	**Scaffold?**	**Audio?**	**Keyframes?**
Standard	36	10	11	no	no	no
Standard+scaffold	29	9	2	yes	no	no
Standard+audio	48	20	7	no	yes	no
Scaffold+audio (best)	**57**	35	3	yes	yes	no
AOI full (default)	45	26	1	yes	yes	yes

Table 19. Gemini 3 Flash per-family success (pass/10), Standard vs. AOI full. The four audio families gain +16 pp in aggregate; the shortfall vs. Gemini 2.5's +48 pp is concentrated in the Dashboard and Transient regressions.

Mode	Pod	Meet	Scr	Car	Dash	Tran	Phn	Int	Col	Gam	**Total**
Standard	0	2	4	4	**8**	3	1	7	5	2	36
AOI full	**6**	**6**	**6**	**5**	1	0	**6**	**8**	5	2	45
Δ	+6	+4	+2	+1	−7	−3	+5	+1	0	0	+9

F DynaCU-Bench Task List

Each of the 100 tasks is identified by a category letter (A–J) and a difficulty-indexed ID. Easy tasks: E1, E2, E3. Medium tasks: M1, M2, M3, M4. Hard tasks: H1, H2, H3.

Example tasks by category:

- **A-E1** (Podcast, Easy): Listen to a podcast segment and identify the guest's name.
- **B-M2** (Meeting, Medium): During a meeting with slides, count the number of items in a presented chart.
- **C-H1** (Video, Hard): Watch a terminal recording and describe the full workflow demonstrated.
- **D-M4** (Carousel, Medium): Identify a specific product name from a rotating carousel.
- **E-E3** (Dashboard, Easy): Read the current warning level from a live-updating dashboard.
- **F-E2** (Transient UI, Easy): Click "Accept" on a cookie consent banner before it auto-dismisses.

Table 20. Keyframe causal probe on Gemini 3 (50 visual-active tasks), holding audio and scaffold fixed and varying only the keyframe channel. Replacing keyframes with same-size noise images reproduces the penalty, and the cost does not scale with count or position—pointing to image-token dilution. Conditions are pairwise indistinguishable ($p \geq 0.375$ at N=50).

Mode	Pass / 50	Rate	What it tests
No keyframes (anchor)	**24**	48%	—
Budget 1 keyframe	20	40%	does count matter?
Budget 2 keyframes	18	36%	
Budget 3 keyframes	20	40%	
Default (budget 5)	18	36%	
Noise images	17	34%	content vs. presence
Duplicate-screenshot images	20	40%	content vs. presence
Keyframes after screenshot	22	44%	position effect

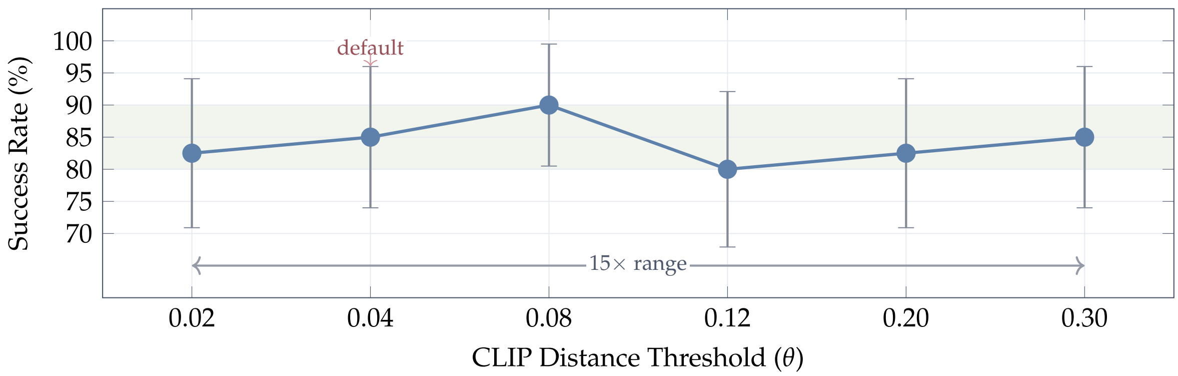


Figure 13. CLIP threshold (θ) sensitivity on 40 visual tasks (categories C–F) with Claude Sonnet 4.6. Error bars are 95% Wilson confidence intervals at N=40 (±9.5–12.1 pp). Point estimates remain within an 80–90% band (shaded) across a 15× range of θ, but the CIs overlap heavily, so we cannot distinguish individual thresholds at this sample size. θ values are placed at uniform spacing on the axis (not to scale). The bimodal distribution of CLIP distances—near-zero for unchanged content, large for meaningful changes—makes the method naturally insensitive to θ.

- **G-M1** (Phone, Medium): Follow spoken instructions during a simulated phone call.
- **H-H2** (Interview, Hard): Complete a multi-question spoken interview with scoring.
- **I-M2** (Collab, Medium): Resolve a conflict in a collaborative document when a remote edit arrives.
- **J-E2** (Games, Easy): Complete 3 rounds of a simple reaction-time game.

All tasks are implemented as self-contained HTML files with embedded JavaScript for task logic, audio synthesis, and evaluation. The complete benchmark (100 HTML files, evaluation scripts, and task definitions) is released publicly with the code.

G CLIP Threshold Calibration

The default threshold $\theta = 0.04$ was calibrated empirically. Web UI events such as modal dialogs appearing produce cosine distances of ∼0.08; video scene cuts produce distances of ∼0.15–0.25; loading spinners and cursor blinks produce distances of ∼0.005–0.02. Setting $\theta = 0.04$ captures all meaningful UI changes while filtering most periodic noise.

Figure 13 shows the CLIP threshold θ sweep across a 15× range from 0.02 to 0.30, evaluated on the 40 visual tasks (categories C–F) where keyframe selection is relevant. Point estimates remain within an 80–90% band throughout, demonstrating that for practitioners who choose the CLIP-based variant, threshold selection is not critical. There is no cliff: even at $\theta = 0.30$ (very aggressive filtering that captures only major visual changes), the point estimate remains at 85%. At $\theta = 0.02$ (very sensitive, capturing small changes), it is 82.5%. The point estimate peaks at $\theta = 0.08$ (90%), but with N=40 the 95% Wilson confidence intervals on every point span ∼10–12 pp and overlap heavily—we therefore do not claim a true sweet spot, only that the method is robust across a 15× range of θ and that any reasonable choice will work. This insensitivity arises because CLIP embeddings produce a bimodal distribution of distances: semantically unchanged frames cluster near zero, while meaningful changes produce large distances, with relatively few samples in between.

H Glossary of Observation Modes

Table 21. Every observation mode used in the paper. KF = inter-step keyframes (with selection strategy); Audio = transcripts of system audio; Narr. = CU-model visual narration persisted as text; Elem. = DOM element list in the prompt; Traj. = structured prior-step trajectory wrapping. **Standard** and **AOI full** run on all models; the rest are Claude Sonnet 4.6 ablations except where a section indicates a Gemini 3 or open-source diagnostic.

Mode	KF	Audio	Narr.	Elem.	Traj.	Where
Standard	—	—	—	—	✓	Table 2
Minimal prompt	—	—	—	—	—	App. B
+element list	—	—	—	✓	—	App. B
+format (scaffold)	—	—	—	✓	✓	§6
Standard+audio	—	✓	—	—	✓	App. E
Selection variants	variant	—	—	✓	✓	§6.2
+keyframes (visual)	CLIP	—	—	✓	✓	§6.2
+ASR	CLIP	✓	—	✓	✓	§6.2
Scaffold+audio (no KF)	—	✓	✓	✓	✓	§6.4
AOI full	pixel gate	✓	✓	✓	✓	Table 2
Narration discarded	pixel gate	✓	not kept	✓	✓	§6.3
Keyframe probes	modified	✓	✓	✓	✓	App. E

I Implementation Details

Software. Python 3.11, Playwright 1.49 for browser automation, PulseAudio 16.1 for audio I/O, edge-TTS for high-quality speech synthesis, OpenAI CLIP ViT-B/16 for keyframe extraction, Whisper large-v3 for speech transcription, vLLM 0.19.0 for local model serving.

Hardware. All experiments run on a single machine with an NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM), 192 GB RAM. Cloud models (Claude, GPT-5.4, Gemini 2.5 Flash) are accessed via HTTP APIs. Local models (EvoCUA-32B, Fara-7B) are served via vLLM with 4-bit quantization (`bitsandbytes`). Whisper runs on CPU to avoid GPU contention. Running the two local models in the `standard_structured` mode (Section 6.1) required patching vLLM 0.19.0 around an NVML/userspace driver mismatch on our host: the CUDA platform

Table 22. DynaCU-Real-Local: Claude Sonnet 4.6 Standard vs. AOI full on 12 real-recording tasks. Both modes tie at 11/12 on this set; the failure is the same task (`R_cast2_pip`) for both.

Mode	Podcast	Meeting	Screencast	Voice	Total / 12
Standard	3/3	3/3	2/3	3/3	**11** (92%)
AOI full	3/3	3/3	2/3	3/3	**11** (92%)

plugin trusts `pynvml.nvmlInit()` alone, and an overly strict free-memory assertion fired when the Whisper service released GPU memory during startup profiling. Both patches are released with the code.

Hyperparameters. Screen capture rate: $\sim$3 Hz. Pixel change threshold (α): 1%. CLIP distance threshold (θ): 0.04. Maximum keyframes per step: 5. Audio capture: a 70 s ring buffer at 16 kHz mono holds rolling history; when the volume gate fires, Whisper is invoked once per step on the new inter-step audio (typically $\sim$3.5 s) extended by a $\sim$3.5 s overlap into the previous interval ($\sim$7 s total) for boundary continuity. Post-action wait: 2.0 s. Maximum steps per task: 15.

J DynaCU-Real-Local Details

The 12 tasks span four sub-domains: 3 podcast (Aesop animal-identification), 3 meeting (Python / Postgres / Rust technical detail), 3 screencast (`git clone` / `pip install` / `npm test` outcome), and 3 voice (yes/no / directions / appointment day). Audio for podcast/meeting/voice tasks is rendered with `espeak` (a formant synthesizer with a deliberately non-neural acoustic profile). Screencast tasks use real `asciinema` v2 cast files generated locally. Source texts are public-domain materials: Aesop's Fables for podcasts, Wikipedia for Python language facts, project documentation for PostgreSQL/MVCC and Rust/tokio. All HTML harnesses, asset-build scripts, and success checks are released with the benchmark.

K Streaming-Baseline Adapter Sanity Check Details

To rule out the explanation that the streaming baselines' weak audio-subset numbers (Section 8) are an artifact of our adapter rather than a model limitation, we evaluate both adapters on a small purely-visual sanity set of five tasks across four categories: C (video / static recording), E (live dashboard), F (transient UI; two tasks, one banner-accept and one cookie-accept subtype), and I (collaborative document). All chosen so the answer can be reached without any audio comprehension. The exact set is C-E1, E-E1, F-E1, F-E2, and I-E1. For each baseline, we open one streaming session per task, send the post-action screenshot as a multimodal turn, and translate any returned tool calls into browser actions through the same path used in the main audio-subset evaluation. The success criterion is the same DOM check as the main benchmark. Table 23 reports the per-task outcome and the modal failure mode where applicable.

Table 23. Per-task adapter sanity-check outcomes for each streaming baseline. Both adapters drive the browser and emit valid tool calls (`click`, `fill`, `type_text`) on every task; every attempted action lands on the DOM. Both score 0/5 task-correct because the models choose the wrong target (e.g., F-E2 cookies dismissed instead of accepted), hallucinate content (C-E1 typed a URL not present in the recording), or commit prematurely (I-E1 typed before reading the page). The same failure modes appear on the audio subset (Table 5), confirming the audio-subset failures are content-driven rather than infrastructural.

Baseline	**C-E1**	**E-E1**	**F-E1**	**F-E2**	**I-E1**	**Total**
OpenAI Realtime adapter (gpt-4o backbone)	✗	✗	✗	✗	✗	0
Gemini Live (2.5 flash native-audio)	✗	✗	✗	✗	✗	0
AOI full (Claude Sonnet 4.6) reference	✓	✗	✓	✓	✓	4